

Deadlock Dungeon: La mazmorra de los recursos

Daniela Rodriguez Reyes

Heidy Valeria Asprilla Rubiano

Profesor

Jamilton Fernando Benavides

Universidad Manuela Beltran

Facultad de Ingenieria de Software

Sistemas Operativos

04 de noviembre de 2026

1. Descripción general del producto

Deadlock Dungeon es un juego web educativo ambientado en una mazmorra. Dos personajes, un caballero y un mago, deben reunir recursos (una llave y un escudo) para abrir la puerta de salida. A medida que el jugador avanza, los personajes se encuentran con situaciones en las que cada uno necesita un recurso que tiene el otro. Si ninguno libera lo que posee mientras espera lo que le falta, se produce un interbloqueo (deadlock) y el juego no puede continuar.

El jugador asigna los recursos, prueba distintas combinaciones, comete errores y observa cuándo se genera un bloqueo y cómo evitarlo. Scripts en Bash y PowerShell analizan cada escenario y determinan si existe un bloqueo o una secuencia segura para que ambos personajes terminen.

2. Planteamiento de la necesidad o problemática

El interbloqueo es uno de los temas de Sistemas Operativos que más cuesta entender cuando solo se estudia con ejemplos teóricos y diagramas estáticos. Conceptos como procesos, recursos compartidos, espera circular y estado seguro resultan abstractos para quienes los ven por primera vez, lo que dificulta identificar por qué se produce un bloqueo y cómo se previene.

3. Justificación de la solución

Un juego permite aprender haciendo. En lugar de memorizar las condiciones del deadlock, el estudiante las vive: asigna recursos, ve el resultado inmediato y entiende la causa del bloqueo. Además, el proyecto integra de forma real los conceptos de Sistemas Operativos, porque el análisis de los escenarios lo realizan scripts de shell y no solo el código de la interfaz.

4. Objetivos

4.1 Objetivo general

Desarrollar un juego web educativo que permita comprender el interbloqueo (deadlock) mediante la asignación interactiva de recursos entre procesos, apoyado en scripts de Bash y PowerShell que analicen cada escenario.

4.2 Objetivos específicos

- Diseñar tres niveles que guíen al jugador desde la asignación básica de recursos hasta la resolución de un bloqueo.
- Implementar una interfaz web con HTML, CSS y JavaScript que represente a los personajes, los recursos y el estado de cada escenario.
- Desarrollar scripts en Bash y PowerShell que detecten bloqueos y verifiquen si existe una secuencia segura.
- Conectar el juego con los scripts mediante un servidor en Node.js.
- Almacenar niveles, personajes, recursos, asignaciones y resultados en una base de datos SQLite.
- Documentar el proyecto y mantener el repositorio organizado en GitHub.

5. Alcance inicial del producto

Incluye:

- Juego web con tres niveles, dos personajes y dos tipos de recursos (llave y escudo).
- Pantalla de inicio, tablero de juego y mensajes de retroalimentación.
- Análisis de escenarios mediante scripts Bash y PowerShell.

- Servidor Node.js que comunica el juego con los scripts.
- Base de datos SQLite para niveles, recursos, asignaciones y resultados.
- Sistema de resultados y puntuación.

No incluye:

- Modificación de procesos o recursos reales del sistema operativo.
- Sistema de usuarios o inicio de sesión.
- Juego multijugador en línea.

6. Usuarios o beneficiarios

Estudiantes de Sistemas Operativos que están aprendiendo sobre interbloqueos, y docentes que quieran una herramienta didáctica para explicar el tema en clase. También sirve a cualquier persona con curiosidad por entender cómo un sistema evita que sus procesos se bloqueen entre sí.

7. Funcionalidades principales

- Selección y avance entre tres niveles.
- Asignación de recursos a los personajes mediante la interfaz.
- Detección automática de interbloqueo por los scripts de análisis.
- Verificación de secuencia segura y mensajes que explican por qué se produjo (o se evitó) el bloqueo.
- Registro de asignaciones y resultados en la base de datos.
- Puntuación y resumen al finalizar cada nivel.

Descripción de los niveles

Nivel	Propósito	Situación
1	Aprender cómo se asignan los recursos	Escenario sencillo sin conflicto entre los personajes
2	Enfrentar un bloqueo	El caballero tiene la llave y necesita el escudo; el mago tiene el escudo y necesita la llave, y ninguno libera su recurso
3	Encontrar una solución	El jugador reorganiza la asignación para hallar una secuencia segura en la que ambos terminen

8. Relación con los conceptos de Sistemas Operativos

Concepto	Cómo se representa en el proyecto
Proceso	Cada personaje (caballero y mago) es un proceso que necesita recursos para terminar su tarea
Recurso compartido	La llave y el escudo, que solo puede tener un personaje a la vez
Asignación de recursos	El jugador decide qué personaje recibe cada recurso
Exclusión mutua	Un recurso en manos de un personaje no puede usarlo el otro

Concepto	Cómo se representa en el proyecto
Retención y espera	Cada personaje conserva su recurso mientras espera el que le falta
Espera circular	El caballero espera al mago y el mago espera al caballero
Interbloqueo (deadlock)	Ninguno puede avanzar y el nivel queda bloqueado
Estado seguro y secuencia segura	Los scripts verifican si existe un orden en el que ambos personajes pueden terminar
Scripting de shell	Bash (Linux) y PowerShell (Windows) ejecutan el análisis de cada escenario

Los scripts son parte central del análisis: reciben la distribución de recursos de un escenario, la evalúan y devuelven si hay bloqueo o si existe una secuencia segura.

9. Diseño de la estructura del producto

9.1 Componentes principales

Componente	Función
Interfaz web	Muestra la mazmorra, los personajes, los recursos y los mensajes; recibe las acciones del jugador

Componente	Función
Servidor Node.js	Recibe las asignaciones del juego, ejecuta los scripts y devuelve el resultado
Scripts de análisis (Bash y PowerShell)	Detectan interbloqueos y calculan secuencias seguras
Base de datos SQLite	Guarda niveles, personajes, recursos, asignaciones y resultados

9.2 Módulos funcionales

- **Módulo de interfaz:** pantalla de inicio, tablero, botones de asignación y mensajes de retroalimentación.
- **Módulo de lógica del juego:** reglas de cada nivel, control del estado y puntuación.
- **Módulo de análisis:** scripts Bash y PowerShell para detectar bloqueos y secuencias seguras.
- **Módulo de comunicación:** rutas del servidor Node.js que conectan el juego con los scripts.
- **Módulo de datos:** consultas y almacenamiento en SQLite.

9.3 Organización de archivos y directorios

```
Deadlock-Dungeon/
│
├── README.md
├── LICENSE
```

```
│
├── documentacion/
│   ├── descripcion_proyecto.pdf
│   ├── requerimientos.pdf
│   ├── arquitectura_inicial.pdf
│   └── diagramas/
│
├── src/
│   ├── frontend/       (HTML, CSS, JavaScript)
│   ├── backend/        (servidor Node.js)
│   ├── scripts/        (Bash y PowerShell)
│   └── database/       (esquema y datos de SQLite)
│
├── recursos/
│   └── imagenes/
│
└── pruebas/
```

> *Ajusta este árbol para que coincida exactamente con las carpetas que ya tienen en el repositorio.*

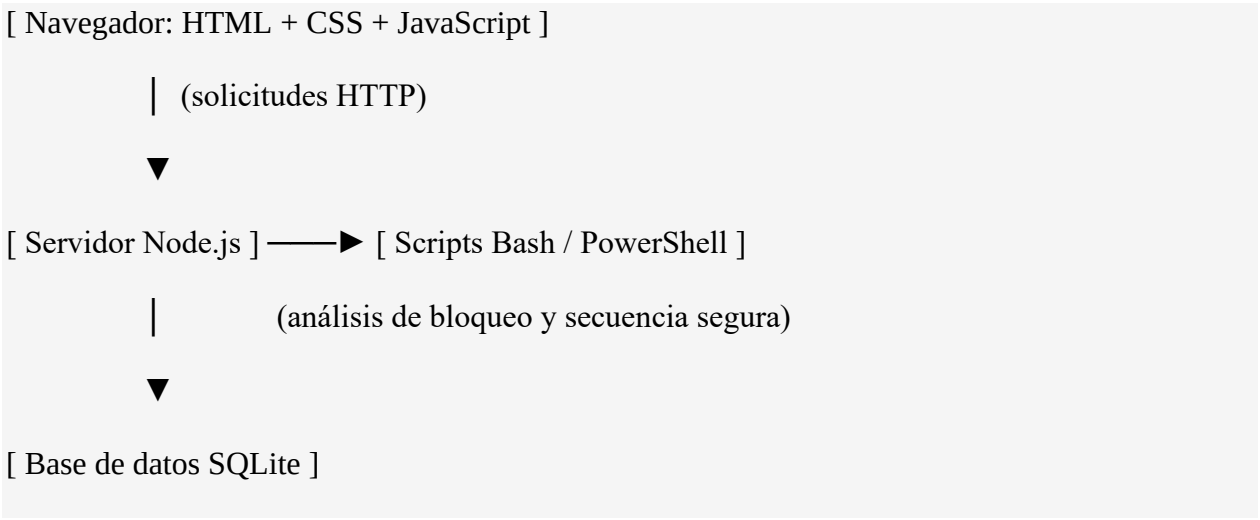
9.4 Tecnologías y herramientas

Tecnología / herramienta	Uso
HTML, CSS y JavaScript	Interfaz visual e interacciones del juego

Tecnología / herramienta	Uso
Node.js	Servidor que conecta el juego con los scripts
Bash	Análisis de escenarios en Linux
PowerShell	Análisis de escenarios en Windows
SQLite	Base de datos de niveles, recursos y resultados
Git y GitHub	Control de versiones y repositorio público
Visual Studio Code	Entorno de desarrollo
Draw.io	Diagramas de estructura y arquitectura

10. Arquitectura inicial de la solución

La solución sigue una arquitectura cliente-servidor sencilla de tres capas:



Flujo básico:

1. El jugador asigna recursos a los personajes en la interfaz.
2. El navegador envía la asignación al servidor Node.js.

3. Node.js ejecuta el script correspondiente al sistema operativo (Bash o PowerShell).
4. El script analiza la asignación y responde si hay bloqueo o una secuencia segura.
5. Node.js guarda el resultado en SQLite y lo devuelve al navegador.
6. La interfaz muestra el mensaje y actualiza el estado del nivel.

> *Aquí inserta el diagrama de arquitectura hecho en Draw.io, igual que el que subieron al repositorio.*

11. Requerimientos

11.1 Requerimientos funcionales

Código	Requerimiento
RF01	El sistema debe mostrar una pantalla de inicio y permitir seleccionar el nivel
RF02	El jugador debe poder asignar la llave y el escudo a cada personaje
RF03	El sistema debe enviar la asignación a los scripts de análisis
RF04	El sistema debe detectar si la asignación genera un interbloqueo
RF05	El sistema debe indicar si existe una secuencia segura
RF06	El sistema debe mostrar mensajes que expliquen el resultado
RF07	El sistema debe registrar asignaciones y resultados en la base de datos
RF08	El sistema debe calcular y mostrar la puntuación por nivel

11.2 Requerimientos no funcionales

Código	Requerimiento
RNF01	La interfaz debe ser clara y fácil de usar para estudiantes
RNF02	El resultado del análisis debe mostrarse en pocos segundos
RNF03	Debe funcionar en Linux y Windows mediante los scripts correspondientes
RNF04	El código y la documentación deben estar organizados en el repositorio
RNF05	La base de datos debe ser local y no requerir instalación compleja

11.3 Requerimientos de hardware y software

Hardware mínimo: computador con 4 GB de RAM, procesador de doble núcleo y 500 MB de espacio libre.

Software: navegador web actualizado, Node.js, SQLite, Bash (Linux o WSL) y PowerShell (Windows), Git y Visual Studio Code.

Entorno de ejecución previsto: Windows y Linux, usando el script que corresponda a cada sistema operativo.

12. Repositorio en GitHub

- **Nombre del repositorio:** Deadlock Dungeon
- **Enlace público:** [pegar enlace]
- **Contenido:** README.md, carpeta documentacion, carpeta src con la estructura inicial, recursos, pruebas y LICENSE.

- **Historial de commits:** evidencia el trabajo de cada integrante desde la creación del repositorio.

13. Estado actual del proyecto y próximos avances

Estado actual: definición del producto, estructura inicial, repositorio creado y documentación base.

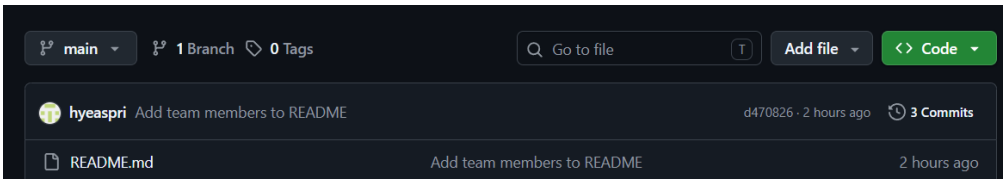
Próximos avances:

- Construir la interfaz del primer nivel.
- Desarrollar los primeros scripts de análisis en Bash y PowerShell.
- Configurar el servidor Node.js y la base de datos SQLite.
- Implementar los niveles 2 y 3.
- Realizar pruebas y validación técnica.

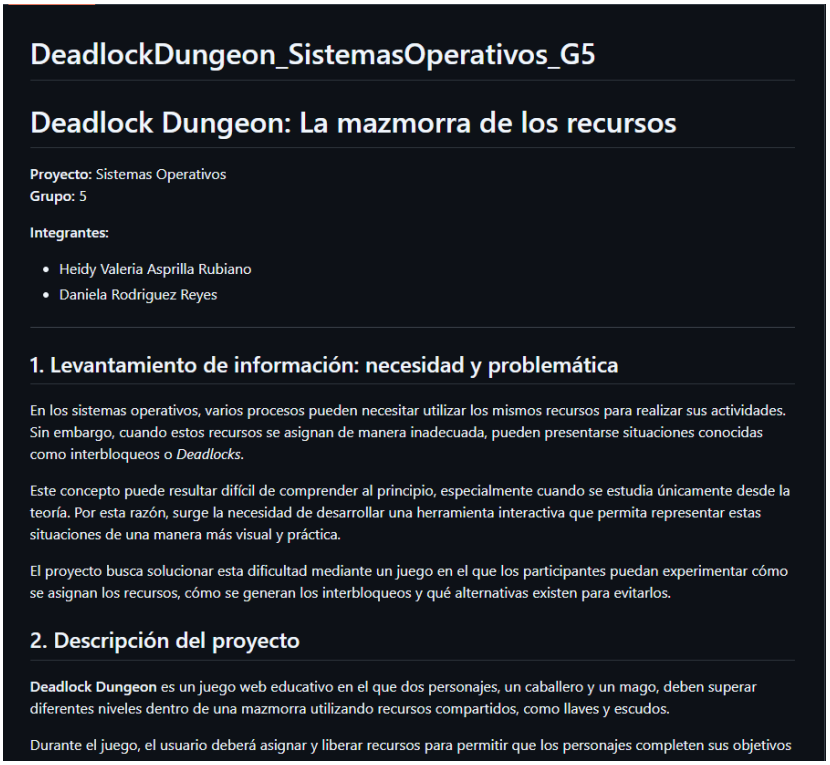
14. Evidencias



Repositorio creado.



Estructura de carpetas y archivos.



DeadlockDungeon_SistemasOperativos_G5

Deadlock Dungeon: La mazmorra de los recursos

Proyecto: Sistemas Operativos
Grupo: 5

Integrantes:

- Heidy Valeria Asprilla Rubiano
- Daniela Rodriguez Reyes

1. Levantamiento de información: necesidad y problemática

En los sistemas operativos, varios procesos pueden necesitar utilizar los mismos recursos para realizar sus actividades. Sin embargo, cuando estos recursos se asignan de manera inadecuada, pueden presentarse situaciones conocidas como interbloqueos o *Deadlocks*.

Este concepto puede resultar difícil de comprender al principio, especialmente cuando se estudia únicamente desde la teoría. Por esta razón, surge la necesidad de desarrollar una herramienta interactiva que permita representar estas situaciones de una manera más visual y práctica.

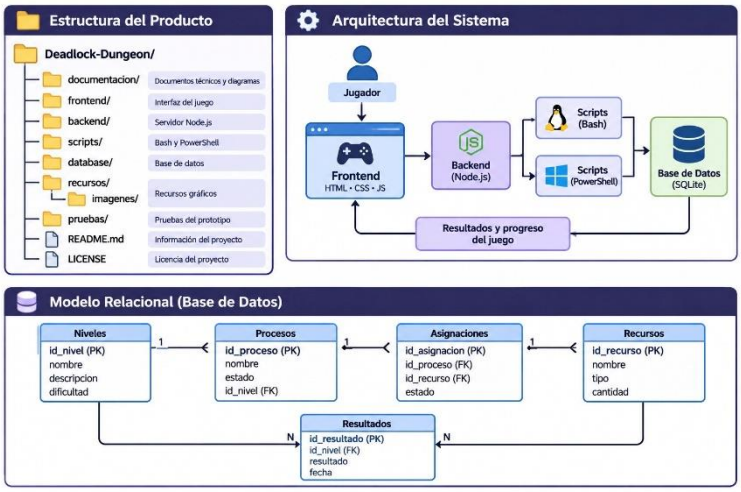
El proyecto busca solucionar esta dificultad mediante un juego en el que los participantes puedan experimentar cómo se asignan los recursos, cómo se generan los interbloqueos y qué alternativas existen para evitarlos.

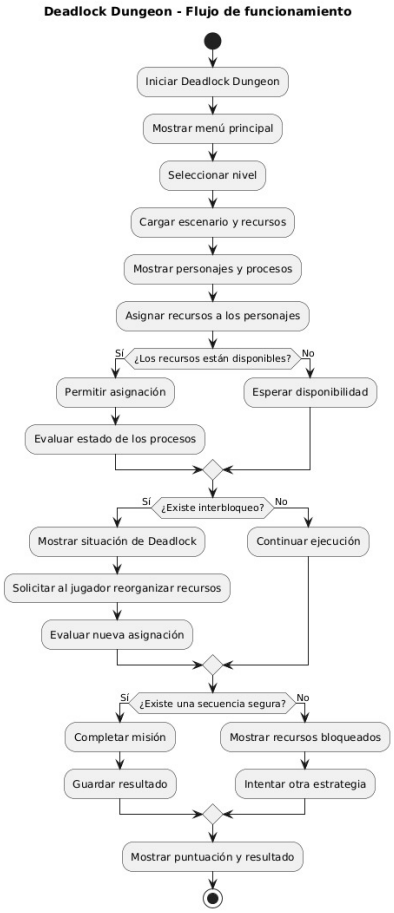
2. Descripción del proyecto

Deadlock Dungeon es un juego web educativo en el que dos personajes, un caballero y un mago, deben superar diferentes niveles dentro de una mazmorra utilizando recursos compartidos, como llaves y escudos.

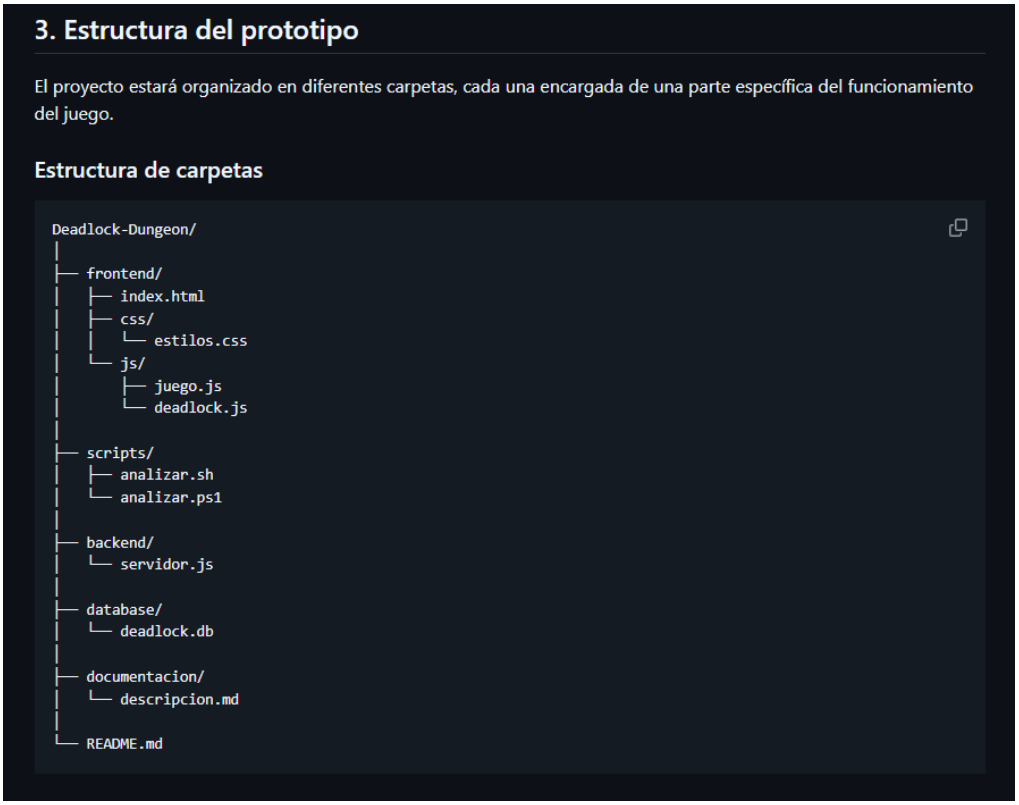
Durante el juego, el usuario deberá asignar y liberar recursos para permitir que los personajes completen sus objetivos

Archivo README.md.





Documentación incorporada.



Diagramas de arquitectura y de estructura del producto.

Link del GITHUB:

https://github.com/hyeaspri/DeadlockDungeon_SistemasOperativos_G5.git

15. Referencias

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10.ª ed.). Wiley.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4.ª ed.). Pearson.
- Documentación oficial de Node.js: https://nodejs.org
- Documentación oficial de SQLite: https://www.sqlite.org/docs.html
- Documentación de GNU Bash: https://www.gnu.org/software/bash/manual/

- Documentación de PowerShell: https://learn.microsoft.com/powershell/